



HIBERNATE

ORM (object Relation Mapping) framework.

ABSTRACT

Hibernate is a powerful and widely-used framework in Java development, offering seamless integration between object-oriented programming and relational databases. This abstract explores the core principles and functionalities of Hibernate, focusing on its role in simplifying database interaction through Object-Relational Mapping (ORM) techniques. We delve into Hibernate's architecture, which facilitates the mapping of Java classes to database tables, streamlining the development process by abstracting away the complexities of SQL queries. Furthermore, we examine Hibernate's persistence mechanism, which provides transparent persistence of Java objects, enabling developers to manipulate data objects without concerning themselves with the underlying database operations. The abstract also discusses key features such as caching, transaction management, and querying capabilities offered by Hibernate, highlighting its significance in enhancing productivity, scalability, and maintainability in Java applications.

Dinesh Sripathi

Trainer, Java Full Stack Web Development

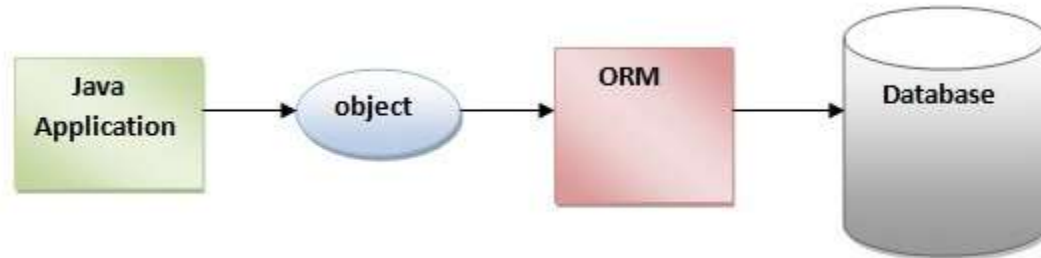
Hibernate

ORM (object Relation Mapping) frameword.

ORM :

It used to link the java object with the database.

It is issue with the JDBC is that it will stick with the certain sql statements. Hence if we change the database then we have to change the queries as well.



Hibernate Framework :

It is used to connect the database with the java.

How to create a project :

1. Here we create the maven project.
2. Then you have to give the group id and artifact id
 - a. **GroupId** : This is where we give the root folder
 - b. **ArtifactId** : This is where we have to give our project name.
3. In maven project we will have the pom.xml and it will contain all the dependencies.
4. To connect the Database we need the jar file. Here we just have to use the dependency code from the maven repository.
5. To get that goto browser and type maven repository.
6. Search for the database connector and copy the maven dependencies and paste them in pom.xml.
 - a. We have to paste them inside the tag dependency.

Syntax :

```
<dependencies>
  <dependency>
    Paste the copied code
  </dependency>
</dependencies>
```

Example :

```
<dependency>
<groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
<version>8.3.0</version>
</dependency>
```

7. Also add the hibernate dependencies. Go to maven repo and search for hiberntate and paste the dependency in pom file.

Example :

```
<dependency>
<groupId>org.hibernate.orm</groupId>
<artifactId>hibernate-core</artifactId>
<version>6.5.0.CR2</version>
</dependency>
```

8. If you are not able to see any maven dependencies then we have to follow the below steps.

- a. Right click on the project
- b. Select maven > right click
- c. Select update maven
- d. Update the maven project it will add the dependencies.

Creating hibernate session :

In hibernate we use the session class from hibernate to insert the data into the database. To create the session we have to follow the below steps.

- a. This session is provided by the session factory, which is a heavy object.
- b. Here we have to configuration the session to the create a connection between session and database.
- c. First, we have to configure to that we have to use the Configuration class.

a. Syntax :

```
Configuration ConfigObject = new Configuration();
```

Example :

```
Configuration config = new Configuration();
```

- b. We have to pass it to configObject.configure();

Syntax :

```
configObject.configure(Hibernate Path Name");
```

Example :

```
config.configure("hibernate.cfg.xml");
```

- c. Then we have to give the credentials as well. We give the credentials in a xml file.
 - i. First create a xml file in src/ main/ resources
 - ii. Select new > other > hibernate
 - iii. Select the cfg hibernate

Example :

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
        <property name="hibernate.connection.password">0000</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/test</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    </session-factory>
</hibernate-configuration>
```

NOTE : If you are not able to see the hibernate option goto market place and search for hibernate tools. Install jboss tool > select hibernate.

- iv. Once it is done. Create the object for the sessionFactory which can get from buildSessionFactory();

Syntax :

```
SessionFactory SFObj = configObject.buildSessionFactory();
```

Example :

```
SessionFactory sessionFactory = config.buildSessionFactory();
```

- v. Then we have to open the session using the openSession method.

Syntax :

```
Session sessionObject = SFObj.openSession();
```

Example :

```
Session s = sessionFactory.openSession();
```

- d. While creating the config file we have to pass the driver class, connection url, username and password.
 - i. In order to insert the data 1st we have to register the class as entity (table) to the main class to save the data.

- 1. To make a class as entity we use annotation.

Syntax :

@Entity

Example :

```
@Entity  
public class Emp
```

- 2. Also register this class in the main java class where you are configuring it. With the help of addAnnotatedClass after configure().

Syntax :

```
configObject.addAnnotatedClass("className.class");
```

Example :

```
conf.addAnnotatedClass(Emp.class);
```

NOTE : In hibernate we must declare at least one column as primary key to do that we have to use the @Id annotation.

- e. Before saving the data we have to get the transaction to save data into database.

Syntax :

```
Session.beginTransaction();
```

Example :

```
session.beginTransaction();
```

- f. We use the session.save(object) to save in database.

Syntax :

```
sessionObject.save(EntityClassObject);
```

Example :

```
session.save(e1);
```

- g. Apart from save session will also have delete, update methods to update the data or to delete it.
- h. In database we perform the all the operations as transaction. Hence we have to commit the operations to save data/ info in database.

Syntax :

```
configObject.getTransaction().commit();
```

Example :

```
session.getTransaction().commit();
```

Example :

```
package com.teks;  
  
import org.hibernate.Session;  
import org.hibernate.SessionFactory;  
import org.hibernate.cfg.Configuration;  
  
public class Application {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        Configuration conf = new Configuration();  
        conf.configure("hibernate.cfg.xml");  
    }  
}
```

```

        conf.addAnnotatedClass(Emp.class);
        SessionFactory sf = conf.buildSessionFactory();

        Session session = sf.openSession();

        session.beginTransaction();
        Emp e1 = new Emp(1, "Tony", 1000.00);

        session.save(e1);

        session.getTransaction().commit();
        System.out.println("Saved...");
    }
}

```

NOTE: To show the sql query we use the property name = “hibernate.show_sql” and set it to true in file hibernate.cfg.xml.

Syntax :

```
<property name = “hibernate.show_sql”>true</property>
```

NOTE : The table name and the entity class must have the same name. If you want to use a different name then we use the annotation called as Table and will provide the name there.

Syntax :

```

@Entity
@Table(name = “tableName”)
Public class className

```

Note : If the column names are different then we use the annotation called column to map it to the main column.

Syntax :

```

@Id
@Column( name = “column name” )

```

Session Object :

- It is a object which will provides us the helper methods. It generated from sessionFactory.
- To generate the sessionFactory we have to do the configuration.
- It doesn’t matter whether you are mentioning the hibernate.cfg.xml file in configuration as the hibernate by default will look for it.

To create/ insert into table :

- We use the session.save(object) to save in database.

Syntax :

```
sessionObject.save(EntityClassObject);
```

Example :

```
session.save(e1);
```

To read into table :

- We use the get method to get the data from database.

Syntax :

```
sessionObject.get ( entityClass.class, PK);
```

- It will return an object for the entity class we have to store the object into the entity class object.

Syntax :

```
EntityClassName EC_Object = sessionObject.get( EntityClass, PrimaryKey );
```

Example :

```
Emp e1 = session.get(Emp.class, 1);
```

NOTE : When you want to read we have to mention the default constructor as well.

- Alternatively we can use the load instead of get();

Syntax :

```
EntityClassName EC_Object = sessionObject.load(EntityClass.class, PK);
```

Example :

```
Emp e1 = session.load(Emp.class, 1);
```

To update Table :

- To update we have to use the session method called as update().

Syntax :

```
EntityClassName EC_Object = sessionObject.update( new EntityClassName( values1, values2,..., valuesn));
```

Example :

```
session.update(new Emp(2, "Wonder Women", 9000));
```

NOTE : We have to use the transactions while updating.

- Alternatively we can use the merge method to update the data.
- To update data for a specific value :
 - To update the data with out changing the entire row then we need to use the below steps.
 1. Load/ get the object to EntityClass
 2. Using the returned object change the value of it.
 3. Then update the object that was changed.

Example :

```
Emp existing = session.load(Emp.class, 1);
existing.setName("maxwell");
session.update(existing);
// session.merge(new Emp(1, "SuperMan", 3000));

session.getTransaction().commit();
System.out.println("Data Updated...");
```

- To delete data from the table :
 - We use the delete() in session to delete the data. Here it will delete based on the primary key most of the time.

Syntax :

```
Session.delete( EntityObject );
```

Example :

- session.beginTransaction();
- session.delete(new Emp(1,"",0.0));
- session.getTransaction().commit();

NOTE: Here the data will update automatically even if you are not updating the data and run the code then the hibernate will follow auto-dirty checking concept.

Auto-Dirty Checking :

Hibernate will check the persistent staged objects whether they are changed or not. If the object value changed by the end of the closing session then it will update the value into the database as well.

Binding the Database to the class :

To bind the database to the class we will use the annotation called as @GeneratedValue() It will help us to bind the primary key to the id of the class. This process is called as reverse binding.

Syntax :

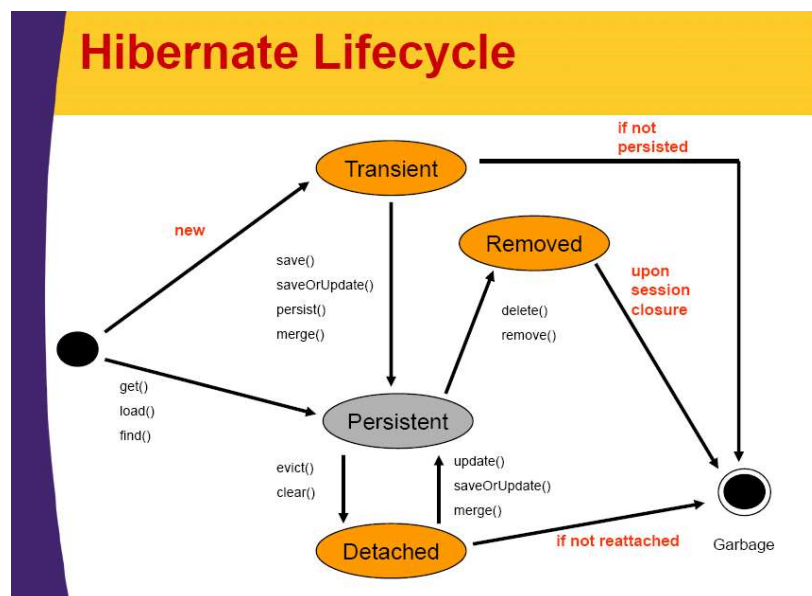
`@GeneratedValue( strategy = GenerationType.IDENTITY )`

Difference Between Get and Load :

GET VS LOAD		
Sno	Get()	Load()
1.	After load() get() is introduced in hibernate as per request.	First load() is introduced
2.	Record returns null	Object Not Found Exception is thrown if record is not present.
3.	Get() always hits database	load() method does not hits the database everytime.
4.	get() method returns either null or fully initialized object.	load() method return proxy object.
5.	As get() always returns fully initialized object there will be less performance.	As load() returns proxy object there will be better performance when compared to get().

States and HQL :

Life Cycle of Hibernate :



1. Transient State :

- The objects which are not associated with the objects are called as transient state.
- You can say these are using before the session.

2. Persistent state :

- The objects which are associated are called as persistent state.

3. Detached State :

- If the object is no longer managing by the persistent state then it will fall under detached state.

NOTE : We can use the `contains()` object to check if the object is associated with the session or not. It will return the Boolean values.

Syntax :

`sessionObject.contains( object );`

To perform the DDL operations in Hibernate :

In hibernate we are going to have a property which is used to perform the DDL operations without creating the table or data. The property name is `HBM2DDL_AUTO` which we need to include in the `hibernate.cfg.xml` file.

Syntax :

```
<property name = "hibernate.hbm2ddl.auto"> options </property>
```

Options :

1. **Update** : used to update the data into database automatically by matching the entity. It will create new columns and constraints if they didn't exist. It will not drop or modify the table.

Example :

```
<property name = "hibernate.hbm2ddl.auto">update</property>
```

2. **Create** : Used to create database schemas from scratch based on entity mapping. It will drop existing schema objects and recreate them if needed.

Example :

```
<property name = "hibernate.hbm2ddl.auto">create</property>
```

NOTE : Before running the main method drop the table

Output :

```
Hibernate: create table Emp_details (id integer not null, salary float(53) not null, DateOfBirth varchar(255), name varchar(255), primary key (id)) engine=InnoDB
```

3. **Create-drop** : It is similar to create but it will also drop table if the sessionFactory is closed.

Example :

```
<property name = "hibernate.hbm2ddl.auto">create-drop</property>
```

4. **None** : This is the default value for the property

NOTE : When you are creating table using the DDL we can control the size of the variables and also the name of the table.

Name : Used for column name

Length : used to mention the size of the datatype of the column.

Nullable : Used to declare whether a column be a not null or not.

Syntax :

```
@Column(name = "col_name", length = size, nullable = false)
```

Embeddable Object :

Here we can use the object as a column for the table. To do that we have to use the annotation called as Embeddable to the subclass.

Example for Entity :

```
@Entity
public class Trail {
    @Id
    private int id;
    private Test t;
    private double salary;
```

Example for subEntity :

```
@Embeddable
public class Test {
    private String name;
    private String email;
```

Example for MainMethod :

```
Test t = new Test("Max", "maxi@email.com");
Trail t1 = new Trail(1, t, 4000);
Configuration config = new Configuration();
```

```

config.configure();
config.addAnnotatedClass(Test.class);
config.addAnnotatedClass(Trail.class);
Session session = config.buildSessionFactory().openSession();
session.beginTransaction();
session.save(t1);
session.getTransaction().commit();
session.close();

```

Hibernate Relations :

Here we are going to have the relations between tables, The types of the relations are

1. *One to One*
2. *One to Many*
3. *Many to One*
4. *Many to Many*

1. Using the OnetoOne annotation :

- a. Imagine we are having two tables and when you want to form a relation then we have to use the annotations for One to One we have to use `@OnetoOne` as it will create a foreign key automatically for the table.

Example for Table 1 :

```

@Entity
@Table(name = "customer")
public class Customers {
    @Id
    @Column(name = "cust_id")
    private int id;
    private String name;
    @OneToOne
    @JoinColumn(name = "order_id")
    private Orders order;
}

```

Example for Table 2 :

```

@Entity
@Table(name = "orders")
public class Orders {
    @Id
    @Column(name = "order_id")
    private int id;
    private String name;
}

```

Example for Main App :

```

Configuration config = new Configuration();
config.configure();
config.addAnnotatedClass(Customers.class);
config.addAnnotatedClass(Orders.class);
Session session = config.buildSessionFactory().openSession();
session.beginTransaction();
Orders o = new Orders(101, "iPhone");
Customers c = new Customers(1, "Ravi");
c.setOrder(o);
session.persist(o);
session.persist(c);
session.getTransaction().commit();
session.close();

```

2. Using one to many :

- Here when we are trying to update or use the table then we have to use the one to many relation.
- It will create a mapping table.
- Also here the tables will get confused as we have to mention the table who will map create the mapping table. To do that we have to use the property called as mappedBy

Syntax :

mappedBy = "EntityClassName"

Customer Table Example ;

```
@Entity
@Table(name = "customer")
public class Customers {
    @Id
    @Column(name = "cust_id")
    private int cust_id;
    private String name;
    @OneToMany(mappedBy = "customer")
    private List<Orders> order = new ArrayList<Orders>();
}
```

Orders Table Example :

```
@Entity
@Table(name = "orders")
public class Orders {
    @Id
    @Column(name = "order_id")
    private int id;
    private String name;
    @ManyToOne
    @JoinColumn(name = "cust_id")
    private Customers customer;
}
```

3. ManyToMany :

- Here we will use to represent the many to many relation between tables.

Syntax :

@ManyToMany
Collection

Example for the CustomersTable :

```
@Entity
@Table(name = "customer")
public class Customers {
    @Id
    @Column(name = "cust_id")
    private int cust_id;
    private String name;
    @ManyToMany(mappedBy = "customer")
    private List<Orders> order = new ArrayList<Orders>();
}
```

Example for the OrdersTable :

```
@Entity
@Table(name = "orders")
public class Orders {
```

```

@Id
@Column(name = "order_id")
private int id;
private String name;
@ManyToMany
private List<Customers> customer = new ArrayList<>();

```

Example for the main method implementation :

```

Configuration config = new Configuration();
config.configure();
System.out.println("Configuration Completed....");
config.addAnnotatedClass(Orders.class);
config.addAnnotatedClass(Customers.class);

Session session = config.buildSessionFactory().openSession();
session.beginTransaction();

Orders o = new Orders();
o.setId(100);
o.setName("iphone 11");
Customers c = new Customers();
c.setId(3);
c.setName("Mike");
c.getOrder().add(o);

o.getCustomer().add(c);

session.save(c);
session.save(o);

session.getTransaction().commit();
session.close();

```

Fetch Lazy Eager :

We use this method to fetch the linked tables, for a single query execute. It will use the left outer join and will fetch the data.

Syntax :

```

@OneToMany( mappedBy = "Table", fetch = FetchType.EAGER)
Private ClassName object;

```

Example :

```

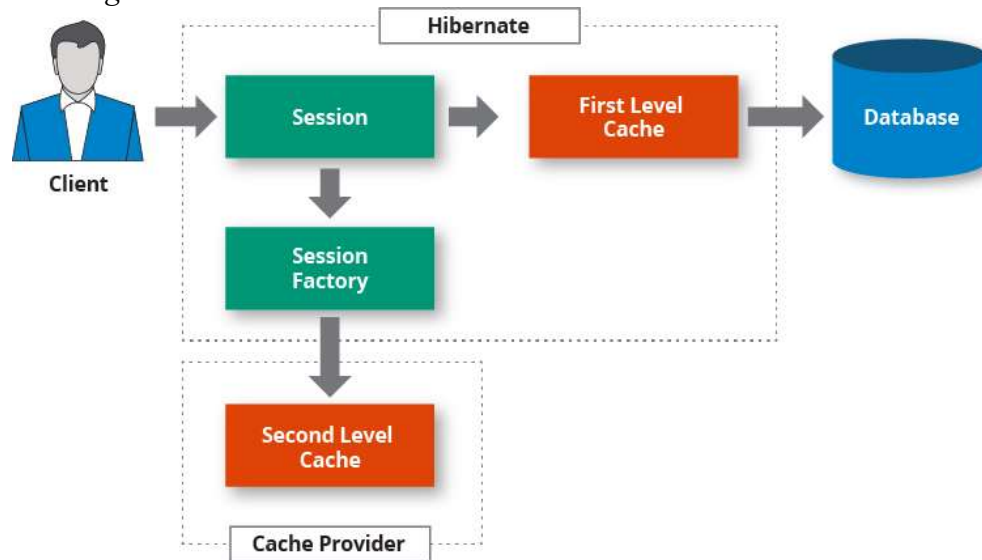
@Entity
@Table(name = "customer")
public class Customers {
    @Id
    @Column(name = "cust_id")
    private int cust_id;
    private String name;
    @OneToMany(mappedBy = "customer", fetch = FetchType.EAGER)
    private List<Orders> order = new ArrayList<Orders>();
}

```

Hibernate Caching :

In hibernate we are having 2 types of cache memory.

1. 1st level Caching
2. 2nd level Caching



a. 1st Level Caching :

Here it will lie between the database and the session. It will store a copy of the data into it when we are fetching the data for the first time in the same session then hibernate will provide the cache memory.

NOTE: In First level cache it will execute the query only for once and the fetched data will be used.

b. 2nd Level Cache :

Here it will not be provided by the hibernate but by the user. It will be shared between the sessions so that all the sessions can share the data between them. But we have to configure some fields.

i. Configuration Steps :

- Configure the pom.xml :
- Configure the ehcache :
- Download the hibernate ehcache configuration
- Configure the hibernate.cfg.xml

1. Use the property name

```
<property name="hibernate.cache.use_second_level_cache"></property>
```

2. Set the property to true.

3. Also we have to use the property name

```
<property name="hibernate.cache.region.factory_class">org.hibernate.cache.ehcache.EhCacheRegionFactory</property>
<property name="hibernate.cache.use_query_cache">true</property>
```

v. Change the entity with the @Cache, @Cacheable

Hibernate Query Language :

Here we can write our own queries in hibernate. To run the query we use the method called as createQuery(). The difference between a normal SQL query and the HQL is the case-sensitive.

createQuery :

It will return a query object after running the query.

Syntax :

```
sessionObject.createQuery(" hql query here");
```

Example :

```
Session.createQuery(" SELECT E from Emp_details E ");
```

Instead of declaring all the values we can use the entity name like as shown below :

Syntax :

```
sessionObject.createObject( "From TableName" );
```

Example :

```
String hql = "from Emp_details";  
Query q = session.createQuery(hql);
```

Selecting the specific columns :

Whenever we are trying to fetch the record it will return the collection of objects hence we have to store it inside the object array.

Syntax :

```
Query queryObject = session.createQuery(hql);
```

Example :

```
Query q = session.createQuery("select name, salary from EmpDetails E Where name = 'Mini' and salary > 3000");  
  
Object[] e = (Object[]) q.uniqueResult();  
  
if(e != null) {  
for(Object e1 : e) {  
    System.out.println(e1);  
}  
}
```

Imagine where we are trying to get the set of data instead of returning a single object it will return a collection of objects. Then we have to store them inside a list.

Example :

```
Query q = session.createQuery("select name, salary from EmpDetails E Where name = 'Mini' or salary > 3000");  
  
List<Object[]> e = (List<Object[]>) q.list();  
  
for(Object[] obj : e) {  
    System.out.println(obj[0] + " " + obj[1]);  
}
```

Performing Aggregate ops :

In hibernate we can perform the aggregate operations like (min, max, avg,...). As it will return a single record then we have to store it inside the list and we use it.

Syntax :

```
sessionObject.createQuery(" SELECT AggOps(colName) from EntityName;  
List listObject = queryObject.list( );  
System.out.println( listObject );
```

Example :

```
Query q = session.createQuery("select max(salary) from EmpDetails E Where name = 'Mini'");  
  
List l = q.list();  
System.out.println(l);
```

To provide the data dynamically we can use the colon or question mark before the variable.

Syntax :

Where colName = :variable;

Example :

```
Query q = session.createQuery("select salary from EmpDetails E Where name = :check");
q.setParameter("check", "Maxi");
List<Double[]> l = q.list();
System.out.println(l);
```

NOTE : Before doing that we have to set the parameter for the variable to do that we have to use the method in query.

Syntax :

```
queryObject.setParameter( key, value );
```

SQL queries in Hibernate :

We can use the sql queries in hibernate to do that we have to create an object for the sql and we have to use the method called as createNativeQuery().

Syntax :

```
NativeQuery sqlObject= sessionObject.createNativeQuery( sql_query );
```

But it will return us an object as we don't want the object instead of that we need the entity object. To get the entity object we use the addEntity() method.

Syntax :

```
sqlObject.addEntity( EntityClass.class );
```

Example :

```
NativeQuery sql = session.createNativeQuery("select * from emp_details where salary = 2000");

sql.addEntity(EmpDetails.class);

List<EmpDetails> emp = sql.list();

for(EmpDetails e : emp) {
    System.out.println(e);
}
```

JPA :

JPA Means Java Persistence API. It is a specification which will help us to change from one ORM to another ORM. Hence we need to use the ORM tools to use the JPA. If we need to fetch data from the database we use the find() method. It will accept us EntityName.class and the id. This find method is actually belongs to interface EntityManager.

Entity Manager :

It is the main control center for managing your entities and interacting with the persistence context. Entity Manager object will be provided by the EntityManagerFactory.

You can create Entitymanager object in two ways.

1. Using session object
2. Using persistence

a. Using SessionObject :

Here we use the session object to create the entityManagerFactory().

Syntax :

```
EntityManagerFactory emf = session.getEntityManagerFactory();
EntityManager em = emf.createEntityManager();
```

Example :

```

Session session = config.buildSessionFactory().openSession();
session.beginTransaction();
EntityManagerFactory emf = session.getEntityManagerFactory();
EntityManager em = emf.createEntityManager();
System.out.println(em.find(EmpDetails.class, 3));

```

b. Using the Persistence :

To use the persistence we use the persistence.xml file to configure. For that we have to follow the below steps.

1. Create a META-INF folder and create a xml file here.
2. Create a persistence xml file.

Example :

```

<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<persistence version="3.0" xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
        https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd">

    <persistence-unit name="jpa">
        <properties>
            <property name="jakarta.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver" />
            <property name="jakarta.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/trail" />
            <property name="jakarta.persistence.jdbc.user" value="root" />
            <property name="jakarta.persistence.jdbc.password" value="0000" />
            <!-- <property name="jakarta.persistence.schema-generation.database.action" value="create" /> -->
            <property name="hibernate.dialect" value="org.hibernate.dialect.MySQLDialect" />
            <property name="hibernate.show_sql" value="true" />
            <property name="hibernate.format_sql" value="true" />
        </properties>
    </persistence-unit>
</persistence>

```

3. Link the persistence xml with the EntityManagerFactory.

Example :

```

EntityManagerFactory entityFactory = Persistence.createEntityManagerFactory("jpa");
EntityManager entityManager = entityFactory.createEntityManager();
System.out.println(entityManager.find(EmpDetails.class, 3));

```